PSD export guidelines

1 Artboards
  1.1 Master configurations
    1.1.1 Pharma ISI
    1.1.2 Pharma preQualifier images
    1.1.3 Tab buttons
    1.1.4 Videos
    1.1.5 Common image assets
  1.2 CTV/OTT
    1.2.1 Images
    1.2.2 Video
    1.2.3 QR Code Config
    1.2.4 Disclaimer
    1.2.5 Custom support
2 Components
  2.1 Tabs
  2.2 Pharma ISI
  2.3 Image
    2.3.1 Assets generator
    2.3.2 Retina images support
  2.4 Mobile Expandable X-Pinning content
  2.5 Text
    2.5.1 How Turbo adjusts text position
    2.5.2 Custom Fonts
  2.6 CTA Button
  2.7 JumpTo Button
  2.8 Video
  2.9 VideoComponent
  2.10 Image Gallery
  2.11 Location Map
  2.12 Comparison Slider
  2.13 Richmedia Gallery
  2.14 Color Picker
  2.15 Hotspot
  2.16 View 360
  2.17 Scrollpane
  2.18 Countdown
  2.19 Nested Tabs
  2.20 Static Product Gallery
  2.21 Dynamic Copy
3 Rendering
4 Exporting
5 Latest supported Photoshop version

Artboards

Photoshop artboards used to define Ad main unit design.

Hide/Show particular artboard(s) in Photoshop will reflect in exported package.

Supported artboard names:

- "970x250" - Desktop InFrame
- "300x250" - Desktop/Mobile InFrame
- "300x600" - Desktop/Mobile InFrame
- "160x600" - Desktop InFrame
- "728x90" - Desktop InFrame
- "970x546" - Desktop Expandable/InStream
- "mobile-instream" - Mobile InStream
- "428x622" - Mobile Expandable
- "master" - common configurations

Master configurations

Common configurations can be set under "master" artboard, such as: tab buttons, peel logo image, preQualifier images, primary/arrow playlist videos and common image assets, like client logo.

Also if you would like to use "Master" feature in VDX Studio Turbo and edit multiple elements across Ad sizes, you should define layers with the same names across Ad size. E.g. "clientLogo" or "button1" or "rmGallery main", etc.

Pharma ISI

To enable Pharma isi you need to add "disclaimer.html" file to the same folder as source PSD file. Subfolders supported.

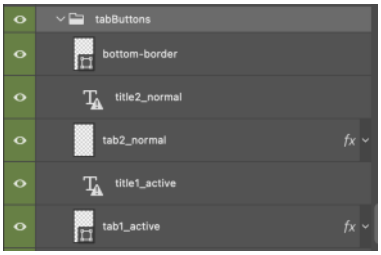
Enabling/disabling isi on teaser can be done in Turbo UI only. Main unit isi can be defined in design with rectangle shape and layer name "isi".

Pharma preQualifier images



Pre-qualifier images should be defined in "master" artboard, "preQualifier_Images" folder and in 2 variants: isi / no isi. E.g. ISI_preQual_160x600.png / preQual_160x600.png.

Tab buttons



To define common configuration for tab buttons, please add "tabButtons" folder under "master" artboard.

"bottom-border" layer will define color and height of tab buttons underline.

Text layers added in specific order will define tab titles and font styles. By adding "_normal", "_active", "_hover" endings, it's possible to customize state text styles, otherwise defaults will be used.

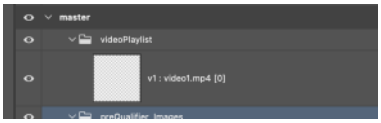
Pixel or shape layers starting with "tab" and having "_normal", "_active", "_hover" endings define background color of tab buttons for specific states.

Also in existing PSD templates we provide extra smartObjects to ease tab buttons preview on all Ad sizes. There are smartObjects "bottom_border" and "navigation_Preview" with "_H" and "_V" endings added into "master" artboard. By modifying these layers and having corresponding smartObject duplications in Ad sizes, it's possible to update design changes for tab buttons in one place - "master" artboard. To achieve this, update design in "tabButtons" folder, then edit corresponding smartObject preview layers and copy/paste design update there.

Peel Logo

To define common peel logo, please add "teaserLogo" pixel or smartObject layer into "master" artboard.

Videos



To define arrow playlist videos, please add "videoPlaylist" folder under "master" artboard.

Each video can be defined in the following format: "{videoId} : {videoFileName} [{posterSecondOffset}]". Posters are generated automatically when package is being uploaded.

Corresponding assets should be placed into the same folder where PSD document is located. Sub-folders are supported.

Order of videos inside "videoPlaylist" folder will be reflected in exported package.

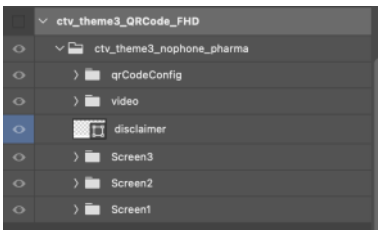
> Videos for the first tab (arrow playlist) must be specified from the start of "videoPlaylist" and the count must be the same across ad sizes.

Common image assets

If you will use smartObject's duplication from "master" artboard across PSD document, you will be able to update such assets in one place.

In such case, VDX Studio Turbo on package upload will check by smartObject ID and pick common image asset, defined in "master" artboard.

CTV/OTT



To define artboard for CTV/OTT assets, please use following naming format "{type}{any text}{quality}":

- type: "ctv", "ott", "ctvott"
- quality: "_HD", "_FHD"

E.g. to generate linked ctv and ott in Full HD quality, you can use "ctvott_theme2_FHD".

To define CTV/OTT theme please add a folder into artboard with following names:

- ctvott_theme2_noqrcode
- ctvott_theme2_noqrcode_pharma
- ott_theme2_noqrcode
- ott_theme2_noqrcode_pharma
- ctv_theme3_iphone
- ctv_theme3_android
- ctv_theme3_nophone
- ctv_theme3_iphone_pharma

- ctv_theme3_android_pharma
- ctv_theme3_nophone_pharma

Images

Assets are defined irrespective of theme.

Most of assets located in corresponding state folders as pixel layers / smartObjects, which will be exported as png images:

- "Screen1" - peelLogo
- "Screen2" - background, foreground
- "Screen3" - tagLine, scanAnchor, bringItAlive, QRCode

Video

raw video asset defined as "video" folder and empty layer with asset filename inside. Corresponding asset file must be located in the same directory with PSD file.

QR Code Config

To define configuration for automatically generated QR Code image, please use "qrCodeConfig" folder:

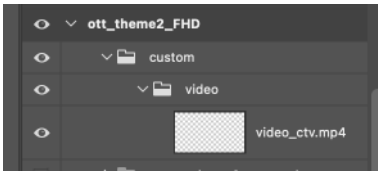
- brandingLogo - pixel/smartObject layer for branding logo image
- colorCode - rect shape to define QR Code color
- url - optional text layer with custom QR Code url as content

Disclaimer

In case of Pharma CTV/OTT, please use "disclaimer" pixel/smartObject layer inside theme folder. It will be generated as png image while exporting.

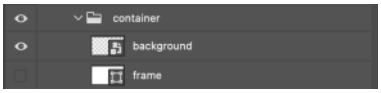
Custom support

To export already generated CTV/OTT video(custom flow), please use "custom" subfolder and use the same "video" structure.



Components

There are base components, like "image" or "text", and complex ones defined with folder(s) structure.



Nesting support we also achieve by using folders, which define "container" base component.

In Photoshop, folder does not contain correct position/bounds information, it is mandatory to provide extra "frame" rect shape layer for this.

"frame" layer also provides such styles:

- background color
- rounded corners
- border color/width
    - border style is not supported, as Photoshop border styles is not fully compatible with web, defined by css
    - if you need custom style like dashed, please make it a part of "background" layer.

If you would like to have a container with background image, it's possible to add extra "background" pixel/smartObject layer.

Such layer has it's own position/bounds which can be used in synergy with "frame" layer. E.g. you would like to add single design element, like arrow to cta button, which is positioned after the text.

Tabs

Tabs component consists of 2 elements:

- "tabButtons" - container for triggers.
- "tabComponent" - container for tabs content.

If "master" configuration is not defined, tabButtons must be defined with the same layers structure for each artboard, otherwise only "frame" layer is required to define position and bounds for each Ad size.

"tabComponent" element must be defined even for single tab "Standard campaigns". It has simple nested containers structure:

- tabComponent
    - tab1
        - frame
        - (background)

- (tab2)
- (tab3)
- frame
- (background)

> For Mobile Expandable artboard using "background" layer to define tabs background image may be problematic because of horizontal responsiveness.
>
> In this case, please define bg image as simple pixel/smartObject with any name, e.g. "bg". There will be "center" pinning logic applied for such element and behaviour on different devices will be correct.

Pharma ISI

Pharma ISI component for main unit can be defined inside each tab, as well as single overlapped container before tabComponent, e.g. for Mobile Expandable such structure is mandatory, as it's sticky to the top.

Image

Image is the base component and can be defined as single pixel/smartObject layer with any name.

While exporting, PSD plugin gets raw pixel data from this layer and generates 8bit png image with following naming format: "{layer name}-{layerID}.png".

For main unit's images the same common image assets rules are applied.

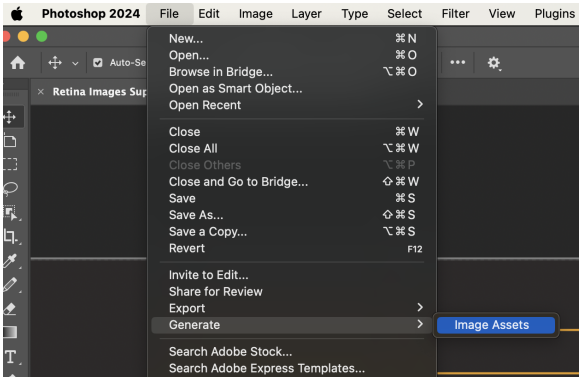
Assets generator

Generating images from raw pixel data is resources consuming and limited to 8bit png images. Instead, it is preferable to use File > Generate > Image Assets.

Please make sure it is marked as enabled, then you can use following naming format for image elements: "{current layer name} + {any valid assets generator string}". E.g. to export cta button background as svg, please use such layer name "background + cta970x546.svg". Full list of supported features can be found here.

Assets generator exports images into the same directory with overwrite. Please make sure, you're using unique filenames.

For png images, please use .png8 ending to export as 8bit images to have optimal file size.



Retina images support

In some cases there is need to have more accurate images, e.g. image contains lines, which normally will be pixelated. Also some Pharma clients requires all the images to be 2x / retina compatible.

Assets generator supports image scaling, but unfortunately it upscales with quality loss, only downscale makes sense.

Layers pre-processing by PSD plugin possible and were implemented initially, but it consumes lots of resources while exporting.

Because of all above reasons, it's mandatory to create extra layer for retina size manually.

To use retina image size, please duplicate a layer, name it exactly the same with " 2x" ending and hide. E.g. "background" and "background 2x" / "clientLogo" and "clientLogo 2x". Also please make sure 2x layer has higher quality pixels data, it can be achieved by using Photoshop smartObjects.

Mobile Expandable X-Pinning content

Classic way
To pin content to the left or to the right side of the window wrap content into a folder, and name it "x-pinning-left" or "x-pinning-right" correspondingly.

Modern way
Add suffix [layout-{xxx}] to the layer name. Available options:

- [layout-start] — will maintain offset to the left edge, and keep element's width fixed
- [layout-center] — will maintain offset to the center, and keep element's width fixed
- [layout-end] — will maintain offset to the right edge, and keep element's width fixed
- [layout-responsive] — will make offsets and width responsive to their container
  * *might produce unexpected results when used in subtree*
- [layout-offset] — will maintain offsets to both edges, but element's width will be flexible
  * *might produce unexpected results when used in subtree*

Text

Text is the base component and can be defined as single text layer.

Currently the following text styling is supported:

- font family
- font size
- text color
- line height
- letter spacing
- all caps / small caps
- super / sub scripts
- text decorations underline / strikethrough
- text align

Partial text styling is supported as well, e.g. you can select part of text and set different font size.

> Photoshop renders text differently than web browsers, it's not possible to have 100% accuracy while exporting.

Please use the following recommendations for more accurate text rendering in web:

- Make sure you have installed used font for text layer and layer updated(you can click and try to edit a layer to sync it).
- Prefer manual hyphenation by adding new line.
- If text elements looks vertically aligned in Photoshop, but it is not in web browser after exporting
  - set anti-alias property to "Crisp" or "None"
  - If pixel perfect adjustments required, you can add it manually to text layer's name, e.g. "text [shiftX: -1, shiftY: 2]"
  - cta buttons supports "center-aligned" keyword, e.g. you can specify layer name as "button1 - center-aligned"
    - Turbo UI now has checkbox to control center text alignment as well as shift X/Y

How Turbo adjusts text position

Photoshop provides pixel bounds for text layers:



Web browsers use font's metrics to render, in details there is good article to read: https://iamvdo.me/en/blog/css-font-metrics-line-height-and-vertical-align

In short, here is line-box height, which will affect on vertical text position:

blocked URL

Content area here is full font's height linked to emSquare and it is embedded into font itself.

Line height is provided by Photoshop, if it's set to "Auto", line height will be 1.2 or 1.2em or 120% of font size.

Line height and content area is always aligned to each other and we use default vertical-align css property "baseline".

To get exact pixels distances for specific text(individual sets of characters), we use measureText Web API. We need font and actual bounding box ascenders values here.

We already have position of the top most pixel of text layer, so required offset can be calculated:

(font full height - line height) / 2 - (font ascender - actual ascender).

Custom Fonts

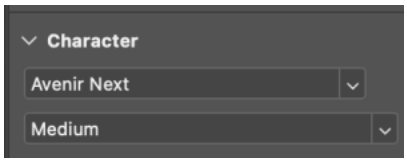
In most cases custom fonts are used. Currently we support ttf, otf, woff and woff2 fonts. WOFF fonts are preferable, as they are optimized for web.

These fonts files must be provided in exported package. This can be done either by putting such files into the same directory as PSD document, either by installing fonts into system(not applicable for woff/woff2).

Subfolders like "fonts" supported.

Photoshop plugin reads font files metadata and detects used font family, but this feature is not supported for woff2 fonts. Woff2 font filenames should have the following format:

"{font family} {font style}.{font type(extension)}", e.g. "Avenir Next Medium.woff2".



CTA Button

CTA Button is a "container" component with text, with the following structure:

- **button{any unique identifier}**, e.g. "button1", "button 970x250 buy now"
  - text layer with any name
  - "frame" layer to define position/bounds and other styles of cta button
  - optional "background" layer, in case bg image is required.

Any other layers inside button folder are ignored.

> CTA url currently not supported and should be added by VDX Studio Turbo simple edit.

JumpTo Button

Jump To Button is a "container" component with text, with the following structure:

- **button-jump [componentId-itemNo]**, e.g. "button-jump [tabComponent-2]", "button-jump [tabComponent-3]"
  - folder (similar to cta-button)
    - "frame" layer to define position/bounds and other styles of cta button
    - optional "background" layer, in case bg image is required
    - any text layer or image layer
  - an image or shape layer

Video

Video is the base component and mandatory to be placed inside tabComponent > tab1 container with name "video".

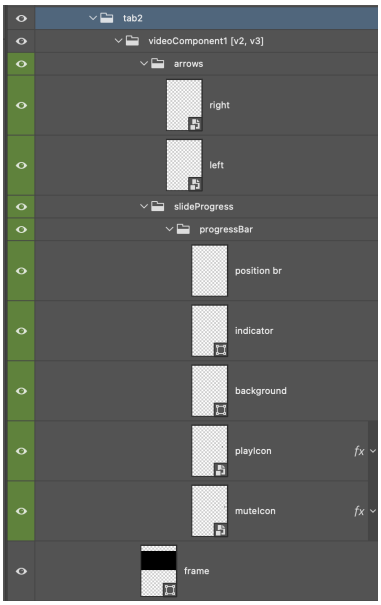
Please check "master" configuration for video playlist information.

To define primary video position/bounds use container structure with "frame" layer or just rectangle shape with the following naming:

video [{videoId}, {videoId}...], e.g. "video [v1, v2]"

VideoComponent

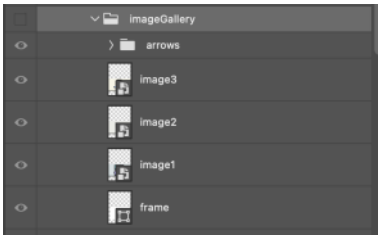
VideoComponent is an advanced video component which is self-sufficient, and doesn't require gallery wrapper.
It might play a single video or a playlist.
Videos are based on "master" configuration for video playlist.



- videoComponent{unique identifier}[{videoId}, {videoId}...] — "videoComponent1 [v2, v3]"
  - (arrows) - optional folder to define playlist arrows
    - right - pixel/smartObject layer for right arrow, position bounds will be used
    - left - pixel/smartObject layer for left arrow, position bounds will be used
  - (slideProgress) - optional folder to define progress styles
    - progressBar - folder to define progress bar styles
      - background - rect shape layer to define bg color of progress bar
      - indicator - rect shape layer to define bg color and width of progress bar indicator
      - playIcon - shape layer to define bg color of play icon
      - muteIcon - shape layer to define bg color of mute icon
      - position {video}|{slide} {tl}|{tr}|{br}|{bl} - empty layer to define controls position
  - "frame" rect shape layer to define position/bounds

Image Gallery

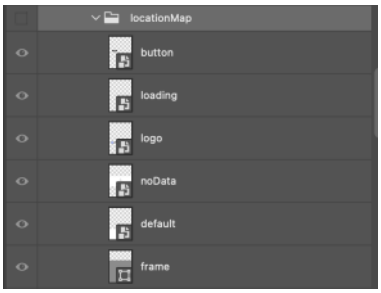
Image gallery is container component with the following structure:



- imageGallery{unique identifier} - "imageGallery1"
  - (arrows) - optional folder to define gallery arrows
    - right - pixel/smartObject layer for right arrow, position bounds will be used
    - left - pixel/smartObject layer for left arrow, position bounds will be used
  - image{unique identifier} - image gallery slide images, order of layers will be used in exported package
  - ...
  - "frame" rect shape layer to define position/bounds

Location Map

Location map is container component with the following structure:



- locationMap{unique identifier} - "locationMap1"
  - popup - rect shape to define popup bounds
  - pin - pixel/smartObject layer to define pin image and it's bounds
  - logo - pixel/smartObject layer to define logo image and it's bounds
  - button - pixel/smartObject layer to define popup button bg image and it's bounds
  - noData - pixel/smartObject layer to define no data image
  - loading - pixel/smartObject layer to define loading image
  - default - pixel/smartObject layer to define default image
  - data - folder to define data JSON file
    - empty layer with data JSON filename, e.g. "data.json"
  - "frame" rect shape layer to define position/bounds

Comparison Slider

Comparison slider is container component with the following structure:

- comparisonSlider{unique identifier} - "comparisonSlider1"
  - preloader - folder layer to define preloader gif
    - empty layer with filename, e.g. "preloader gif", this file should be in source package folder
    - or rectangle shape layer named "bar" to render loading progress line instead of preloader amimation
  - "before"/"left" or "after"/"right" - folder layer to define image/video/rich-media-content to compare
    - empty "video" layer with id link to playlist, e.g. "video [v1]"
      - or
    - pixel/smartObject layer starting with "image", e.g. "image1", "imageBefore.jpg" (assets generator supported)
      - or
    - folder layer "content" with any structure similar to gallery slide

If multiple configurations provided inside compare folders, the following priority will be used: "video" > "image" > "content"

Richmedia Gallery

Richmedia gallery is container component with the following structure:

- rmGallery{unique identifier} - "rmGallery1"
  - (arrows) - optional folder to define gallery arrows
    - right - pixel/smartObject layer for right arrow, position bounds will be used
    - left - pixel/smartObject layer for left arrow, position bounds will be used
  - (slideProgress) - optional folder to define slide progress styles
    - circularBar - folder to define circular progress styles
      - indicator - shape layer to define bg color, size and width of circular progress indicator
      - playIcon - shape layer to define bg color of play icon
      - muteIcon - shape layer to define bg color and position of mute icon
      - frame - rect shape layer to define position/bounds as well as bg color
    - progressBar - folder to define progress bar styles
      - background - rect shape layer to define bg color of progress bar
      - indicator - rect shape layer to define bg color and width of progress bar indicator
      - playIcon - shape layer to define bg color of play icon
      - muteIcon - shape layer to define bg color of mute icon
      - position {video}|{slide} {tl}|{tr}|{br}|{bl} - empty layer to define controls position
  - slide{unique identifier} - folder to define gallery slide content
    - any other valid components layers structure, including video
    - ("frame") optional rect shape layer to define bg color of a slide
    - ("background") optional layer, in case bg image is required for a slide
  - ...
  - "frame" rect shape layer to define position/bounds
  - ("background") optional layer, in case common bg image is required for gallery

- For both slideProgress styles, it's possible to use/edit shadow effect either for playIcon either for muteIcon layers.
- If slideProgress is not provided, autoplay slide animation will be disabled

in case "frame" layers are present in slides as well as in gallery folder, their position/bounds must be exactly the same for correct component behaviour.

Color Picker

Color picker is container component, which extends richmedia gallery with the following structure:

- colorPickerGallery{unique identifier} - "colorPickerGallery1"
  - ... rmGallery layers ...
  - trigger{unique identifier} - folder to define color trigger styles
    - {state}Image - pixel/smartObject layer to define state(normal/active/hover) bg image
    - {state} - optional rect or ellipse shape layer to define trigger bg color and border width/color for specific state(normal/active/hover)
    - "frame" rect or ellipse shape layer to define position/bounds of trigger, it's type, bg color, border width/color

Hotspot

Hotspot is container component, which extends color picker with the following structure:

- hotSpotGallery{unique identifier} - "hotSpotGallery1"
  - slide{unique identifier} - folder to define gallery slide content
    - any other valid components layers structure
    - closeButton - pixel/smartObject layer to define bg image and position/bounds of close button for a slide
    - "frame" rect shape layer to define slide position/bounds, bg color, border width/color and rounded corners of a slide
    - ("background") optional layer, in case bg image is required for a slide
  - trigger{unique identifier} - folder to define color trigger styles
    - {state}Image - pixel/smartObject layer to define state(normal/active/hover) bg image
    - {state} - optional rect or ellipse shape layer to define trigger bg color and border width/color for specific state(normal/active/hover)
    - "frame" rect or ellipse shape layer to define position/bounds of trigger, it's type, bg color, border width/color
  - (backdrop) — optional backdrop shape layer to overlay content behind popup

View 360

Module will be enabled once under hotspot container "sequence" folder is added.
Optionally there might be "arrows" folder to define rotation arrows.

- view360_{unique identifier} — "view360_1"
  - (arrows) — optional folder to define gallery arrows
    - right — pixel/smartObject layer for right arrow, position bounds will be used
    - left — pixel/smartObject layer for left arrow, position bounds will be used
  - (sequence) — contains sequence frame folders, uses "cadre{xx}" naming to avoid collision with "frame" layers.
    - cadre{order_number} — order numbering starts from 1 — contains layers which defines background image, as well as every visible hotspot and popup positions at corresponding frame
      - slide{order_number} — rectangle (same size as corresponding popup size) which defines top/left position of the popup
      - trigger{order_number} — rectangle/circle (same size as corresponding hotspot size) which defines top/left position of the hotspot
      - underlay — background image of corresponding frame (it's size and position is ignored, only image is retrieved to cover the background)
  - ... hotspot layers ...

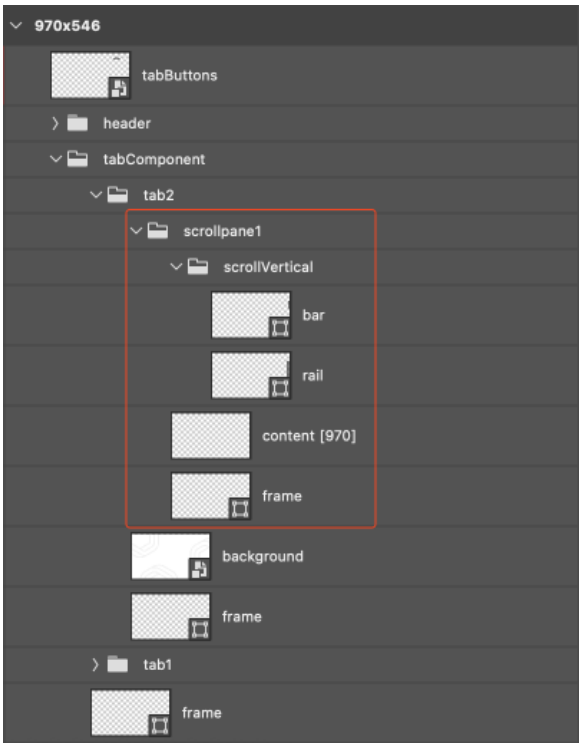
Sample PSD file on the box.

Scrollpane

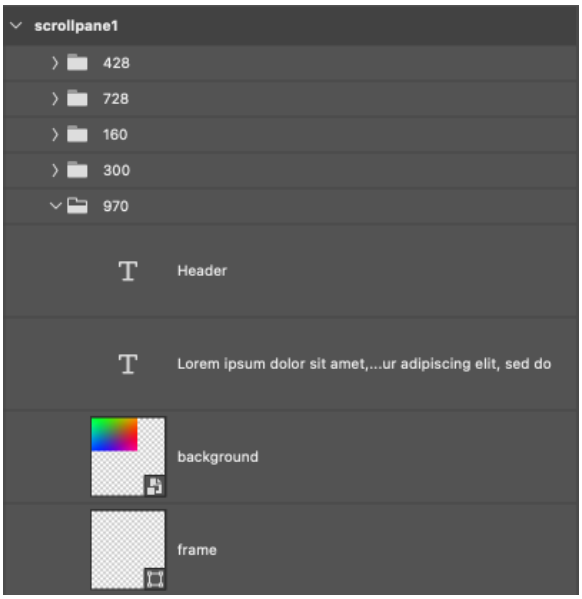
Scrollpane is a container component with following structure:

- scrollpane{unique_id} — for ex. "scrollpane1"
  - (scrollVertical) — optional folder to define vertical scrollbar
    - bar — rect shape layer to define scroll bar offset, width & color
    - rail — rect shape layer to define scroll rail position, width, length & color

    - (scrollHorizontal) — optional folder to define horizontal scrollbar
        - bar — rect shape layer to define scroll bar offset, width & color
        - rail — rect shape layer to define scroll rail position, width, length & color
    - content [{unique_content_identifier}] — empty layer with reference to content from related art board
        - make sure separate art board with content is named same as the component folder itself, i. e. scrollpane{unique_id}
    - (frame) — rect shape layer to define position/bounds of the component



- {unique_content_identifier} - folder to define scrollpane content (ad-specific)
    - any other valid components layers structure
    - "frame" rect shape layer to define slide position /bounds, bg color, border width/color and rounded corners of a slide
    - ("background") optional layer, in case bg image is required for a slide

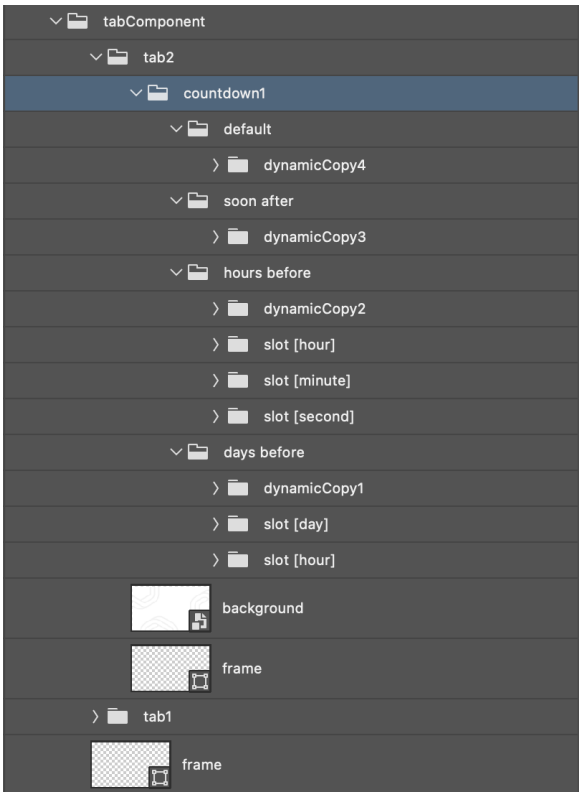


Countdown

Countdown is a container component with following structure:

- countdown{unique_id} — for ex. "countdown1"
    - (any_content_with_clocks) — folder to host content with clock components, supposed to have a target date
        - slot [day] — optional folder which defines "days" count
            - textStyle — text style of the slot
            - frame — bounds of the slot (important for "roll" theme)
        - slot [hour] — optional folder which defines "hours" count
            - textStyle — text style of the slot
            - frame — bounds of the slot (important for "roll" theme)

        - slot [minute] — optional folder which defines "minutes" count
            - textStyle — text style of the slot
            - frame — bounds of the slot (important for "roll" theme)
        - slot [second] — optional folder which defines "seconds" count
            - textStyle — text style of the slot
            - frame — bounds of the slot (important for "roll" theme)
        - any other valid layers structure (including DynamicCopy component)
    - (any_other_content_with_clocks) ...
    - (any_content_without_clocks) — folder to define non-clock content (for ex. to be shown after the target date)
        - valid layers structure (including DynamicCopy component)
    - (any_other_content_without_clocks) ...



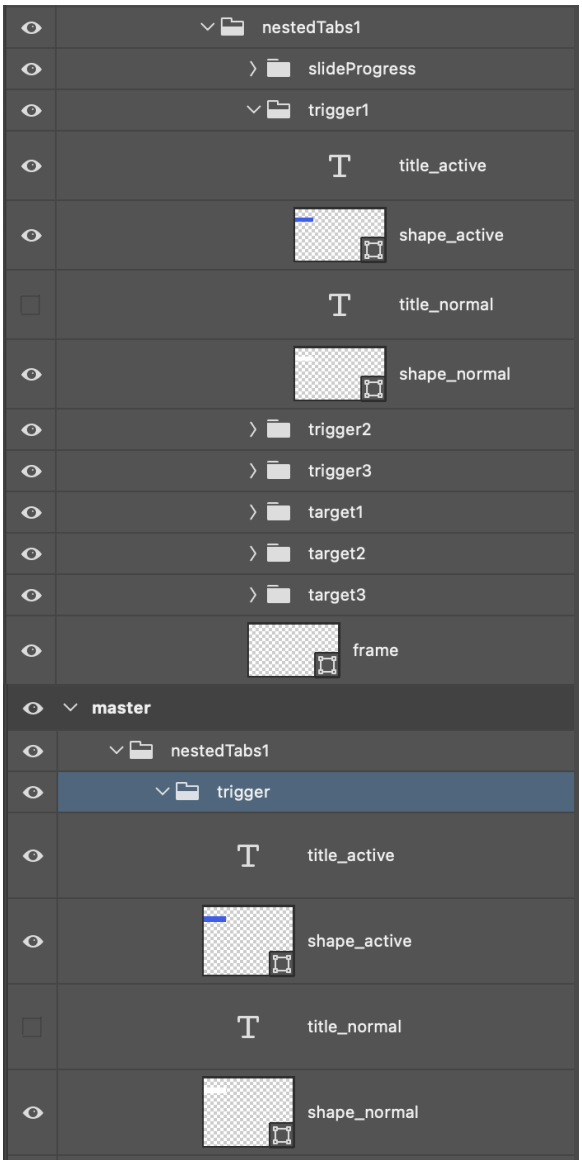
Nested Tabs

Nested Tabs is a container component with following structure:

- nestedTabs{unique_id} — for ex. "nestedTabs1"
    - target{unique identifier} — folder to define tab content
        - any other valid components layers structure
        - ("background") — optional layer, in case common bg image for a slide
        - ("frame") — optional rect shape layer to define bg color of a slide
    - ...
    - trigger{unique identifier} — folder to define trigger button
        - any other valid components layers structure
        - by adding "_normal", "_active" endings, it's possible to customize state styles, otherwise defaults will be used.
        - ("background") — optional layer, bg image for a button
        - ("frame") — optional rect shape layer to define bg color of a button
    - ...
    - ("background") — optional layer, in case common bg image is required for tabs component
    - "frame" — rect shape layer to define position/bounds of the tabs component

Master art-board might contain layers to define trigger buttons common styling

- nestedTabs{unique_id} — for ex. "nestedTabs1"
    - trigger — folder to override every trigger buttons
        - any other valid components layers structure, matched same structure as in size-specific art-board
    - trigger{unique identifier} — folder to override specific trigger button
        - any other valid components layers structure, matched same structure as in size-specific art-board



Static Product Gallery

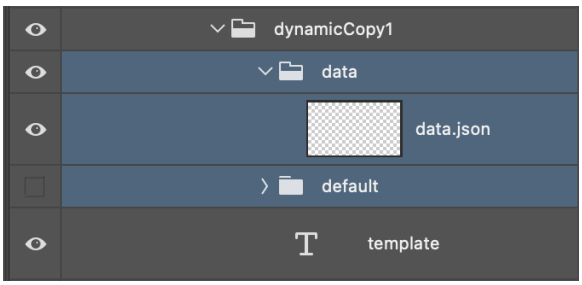
Static Product Gallery is container component with the following structure:

- staticProductGallery{unique identifier} - "staticProductGallery1"
  - (arrows) - optional folder to define gallery arrows
    - right - pixel/smartObject layer for right arrow, position bounds will be used
    - left - pixel/smartObject layer for left arrow, position bounds will be used
  - (slideProgress) - optional folder to define slide progress styles
    - circularBar - folder to define circular progress styles
      - indicator - shape layer to define bg color, size and width of circular progress indicator
      - playIcon - shape layer to define bg color of play icon
      - muteIcon - shape layer to define bg color and position of mute icon
      - frame - rect shape layer to define position/bounds as well as bg color
    - progressBar - folder to define progress bar styles
      - background - rect shape layer to define bg color of progress bar
      - indicator - rect shape layer to define bg color and width of progress bar indicator
      - playIcon - shape layer to define bg color of play icon
      - muteIcon - shape layer to define bg color of mute icon
      - position {video}|{slide} {tl}|{tr}|{br}|{bl} - empty layer to define controls position
  - slide{unique identifier} - folder to define gallery slide content
    - any other valid components layers structure, including video
    - ("frame") optional rect shape layer to define bg color of a slide
    - ("background") optional layer, in case bg image is required for a slide
  - ...

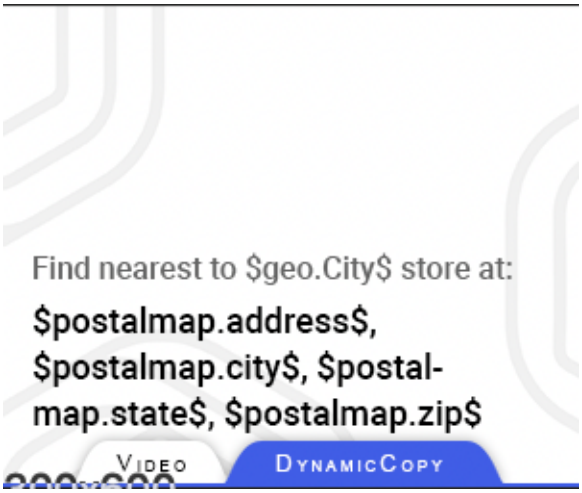
- "frame" rect shape layer to define position/bounds
- ("background") optional layer, in case common bg image is required for gallery

Dynamic Copy

Dynamic Copy is a container component with following structure:



- dynamicCopy{unique_id} — for ex. "dynamicCopy1"
  - data — folder to define data JSON file
    - empty layer with data JSON filename, e.g. "data.json"
  - default — folder to define "default" content to be rendered in case of input data issues
  - any text layer(s)
    - text layer might contain any amount of tokens wrapped into $$ signs
    - for ex "$geo.City$", "$postal.address$" — refer to wiki page (1st column)
  - ("background") — optional layer, in case common bg image is required for tabs component
  - "frame" — rect shape layer to define position/bounds of the tabs component



Text layer type

Make sure to use text boxes in order to properly define text-auto-fit boundaries

Text with Tokens alignment

If text layer uses tokens, then it's pixel-perfect positioning is disabled, since it's content wouldn't match original text from Photoshop.

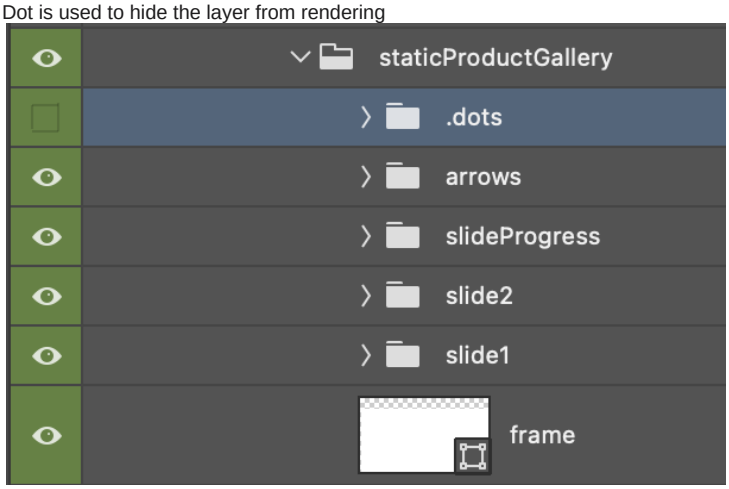
Product Gallery is a container component with following structure:

- productGallery{unique_id} — for ex. "productGallery1"
  - (arrows) — optional folder to define gallery arrows
    - right — pixel/smartObject layer for right arrow, position bounds will be used
    - left — pixel/smartObject layer for left arrow, position bounds will be used
  - (slideProgress) — optional folder to define slide progress styles
    - circularBar — folder to define circular progress styles
      - indicator — shape layer to define bg color, size and width of circular progress indicator
      - playIcon — shape layer to define bg color of play icon
      - muteIcon — shape layer to define bg color and position of mute icon
      - frame — rect shape layer to define position/bounds as well as bg color
    - progressBar — folder to define progress bar styles
      - background - rect shape layer to define bg color of progress bar
      - indicator - rect shape layer to define bg color and width of progress bar indicator
      - playIcon - shape layer to define bg color of play icon
      - muteIcon - shape layer to define bg color of mute icon
      - position {video}|{slide} {tl}|{tr}|{br}|{bl} - empty layer to define controls position
  - (data) — **adsize/master artboard** defines required data input entries (not more than two normally)
    - dataInput — text layer with fine data with predefined json, or evaluated string
    - dataUrl — text layer with fine data url
    - dataFile — folder to define fine data local json file
      - {filename.json} — empty layer named as local json filename
    - dataRawInput — text layer with raw data with predefined json, or evaluated string (should be combined with data-cook)
    - dataRawUrl — text layer with raw data url (should be combined with data-cook)
    - dataRawFile — folder to define raw data local json file
      - {filename.json} — empty layer named as local json filename
    - dataCook — text layer with remapper function raw fine — (rawData, dynamicData) => { return {common:{}, list:[]} }
    - dataCreate — text layer with create async function — async(dynamicData) => { return {common:{}, list:[]} }
  - (grid) — defines slide grid layout
    - gridLayout {columns_count}x{rows_count} — layer's name defines grid layout
    - gridWrapperHtml — **adsize/master artboard** optional text layer which defines grid wrapper element html markup
    - gridCellHtml — text layer which defines product element html markup
    - gridCellSize {auto}|{width_px height_px} — layer's name defines product card size (auto size will fill full slide)
    - gridCenter {center}|{top}|{bottom}|{left}|{right}|{space-between}|{space-around}|{space-evenly}|{stretch} — layer's name defines product cells centering within slide

        - gridGap {length_px} — gap size between product cells
    - (error) — defines error screens
        - noDataHtml — **adsize/master artboard** text layer with "No Data" error screen html markup
    - (css) — custom styling settings
        - cssInput — **adsize/master artboard** text layer with css rules
        - cssFile — **adsize/master artboard** links local css file as an asset

Rendering

Dot is used to hide the layer from rendering



Exporting

PSD plugin has single button to export "Plugins" > "VDX Turbo" > "Export as zip"

> There is known long path issue while exporting from box drive.

While exporting the same directory where PSD file is located will be scanned for assets mentioned in PSD layers structure(videos, fonts, json files, etc.). System fonts locations will be scanned as well:

- "C:\Windows\Fonts" folder on windows
- "/System/Library/Fonts" and "{userdir}/Library/Fonts" folders on MacOS

Latest supported Photoshop version

26.8.1